Lab # 06

COURSE CODE: SE-206L

COURSE TITLE:

Software Construction & Development

LAB TITLE

Abstract Data Types (ADT)

LAB ASSESSMENT TABLE:

Obtained Marks	Total Marks

Submitted To: Engr. Rizwan Shah.

Submitted By: Muhammad Danial.

Registration – No: 24ABSWE0021.

Semester: 5th

Date: 24-09-2026.

Dept. of Software Engineering

UET Peshawar, Abbottabad Campus.

Instructor Sign : _____

Lab 6:

Abstract Data Types (ADT)

Repository Link: https://github.com/danial-uet/Software-Construction-Labs./tree/master/Lab6

Objective

Implement simple ADTs in Java, use interfaces to define contracts, and apply abstraction and encapsulation principles

Learning Outcomes

After completing this lab, students will be able to:

1. **Stack ADT Implementation:** Applied the LIFO principle by implementing concrete Stack operations and validating pop behavior.
2. **Data Encapsulation:** Applied access modifiers and getters to restrict direct field mutation and protect object state.
3. **Programming to Abstractions:** Utilized Java Collections interfaces (List) to decouple code dependencies from concrete classes.
4. **ADT Interface Design:** Designed and structured abstract data types using Java interfaces to separate specification from implementation.
5. **Automated Unit Testing:** Constructed JUnit test cases to systematically validate data structure operations and guarantee functional accuracy.

Lab Task 1 — Implementing the Stack ADT

Objective:

Test the concrete implementation of the Stack ADT based on the LIFO principle.

Implementation

- Use the ArrayStack implementation provided in the Lab Manual.
- Write a test performing the following: push(10); push(20); push(30); .
- Verify that calling pop() returns 30.

Solution

For 1st lab task, one Java Interface & two Java Classes were created by me inside AbstractDataTypes Package in ATDLab Project:

- Stack.java, (Java Interface):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/AbstractDataTypes/Stack.java

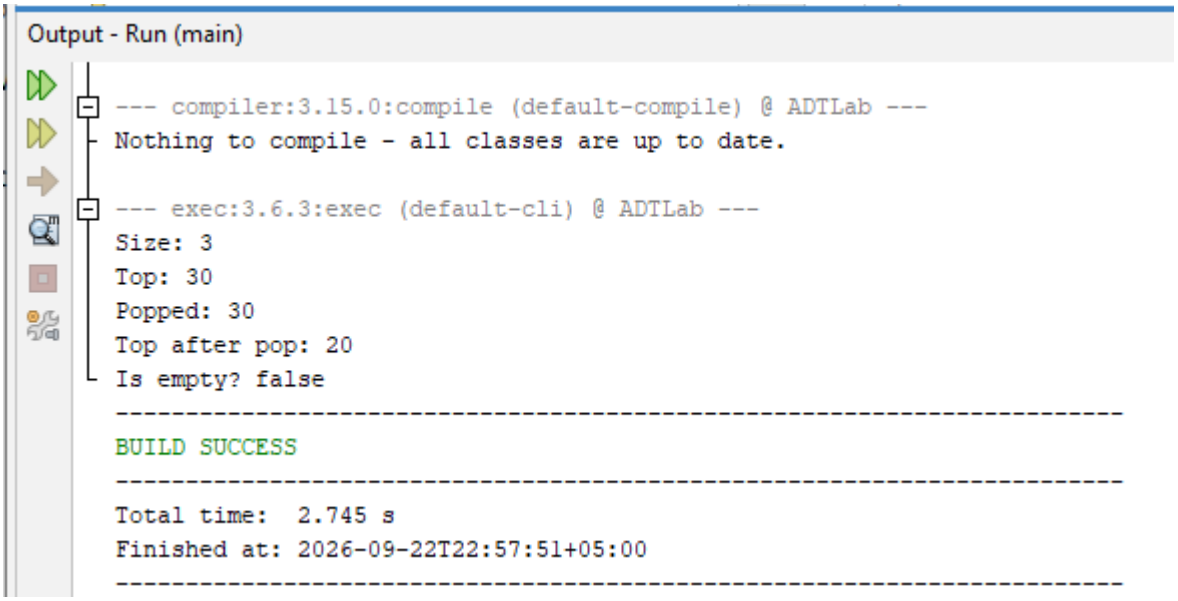
● ArrayStack.java (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/AbstractDataTypes/ArrayStack.java

● main.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/AbstractDataTypes/main.java

Output:



Lab Task 2 — Data Encapsulation

Objective:

Protect internal data by hiding implementation details from the user.

Implementation (Student.java):

• Create a Student class with private fields: id (int), name (String), and cgpa (double) .
• Implement public getter methods (getId() , getName() , getCgpa()) to access the private data.

Test ideas

Attempt to access the variables directly in a main method to observe the compilation error, proving encapsulation works.

Solution

For 2nd lab task, two Java Classes were created by me inside Task2 Package in ATDLab Project:

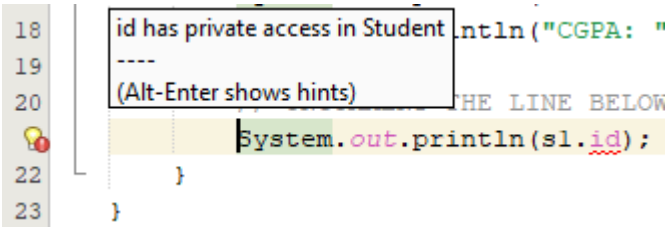
● Student.java, (Java Class):

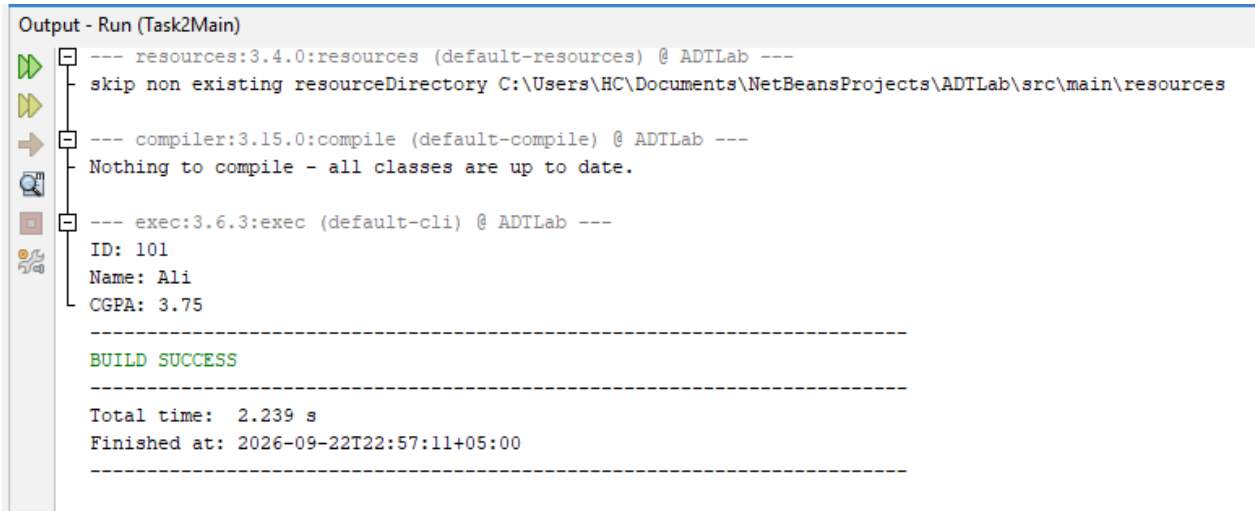
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task2/Student.java

● Task2Main.java, (Junit test class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task2/Task2Main.java

Output:





Lab Task 3 — Programming to an Abstraction

Objective:

Use Java Collections to prove that client code depends on the abstraction, not a specific implementation.

Implementation (Student.java):

- Declare a list variable depending on the interface: List students; .
- Instantiate it as an ArrayList (students = new ArrayList<>();) and add a student ("Ali")
- Reassign the exact same variable as a LinkedList (students = new LinkedList<>();) and add a student.

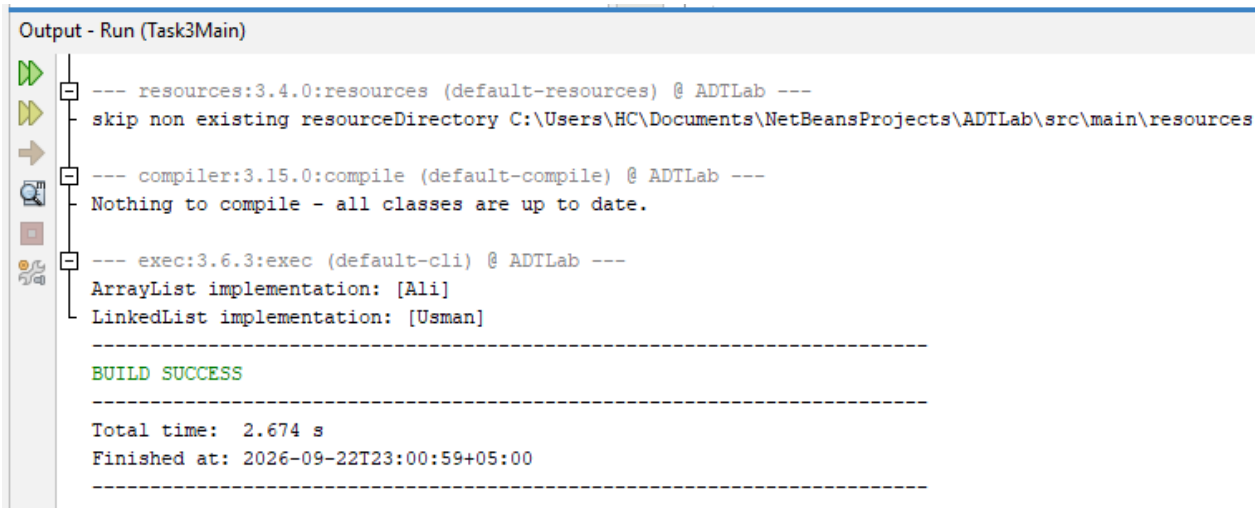
Solution

For 3rd lab task, single Java Class was created by me inside Task3 Package in ATDLab Project:

● Task2Main.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task3/Task3Main.java

Output:



Lab Task 4 — Library System ADT Design

Objective:

Design an ADT for a Library System focusing on WHAT it does, not HOW.

Tasks:

● Identify the data: Book, Book ID, Title, Author.

● Identify operations: addBook() , removeBook() , searchBook() , issueBook() , returnBook().

● Create the Java interface LibrarySystem representing this contract.

● Create a concrete class `LibraryImplementation` that uses a Java Collection (like `ArrayList` or `HashMap`) to implement the interface.

Solution

For 4th lab task, one Java Interface & three Java Classes were created by me inside Task4 Package in ATDLab Project:

● LibrarySystem.java, (Java Interface):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task4/LibrarySystem.java

● Book.java, (1st Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task4/Book.java

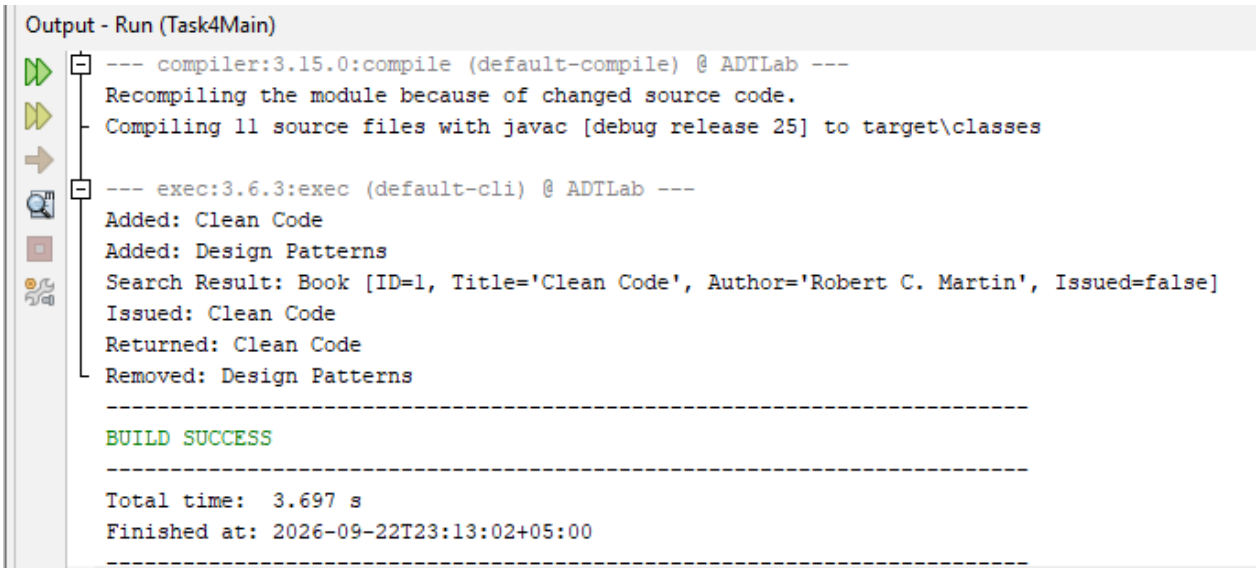
● LibraryImplementation.java, (2nd Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task4/LibraryImplementation.java

● Task4Main.java, (3rd Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task4/Task4Main.java

Output:



Lab Task 5 — Student Management System

Objective:

Separate the specification from the implementation for a student collection.

Tasks:

• Define the StudentCollection ADT with the following operations: addStudent(Student student) , , findStudent(int removeStudent(int id) id) , getSize() , and isEmpty().

• Create a class that implements this interface.

• Write JUnit tests validating the operations.

Solution

For 5th lab task, one Java Interface & two Java Classes were created by me inside Task5 Package in ATDLab Project:

● StudentCollection.java, (Java Interface):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task5/StudentCollection.java

● Student.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task5/Student.java

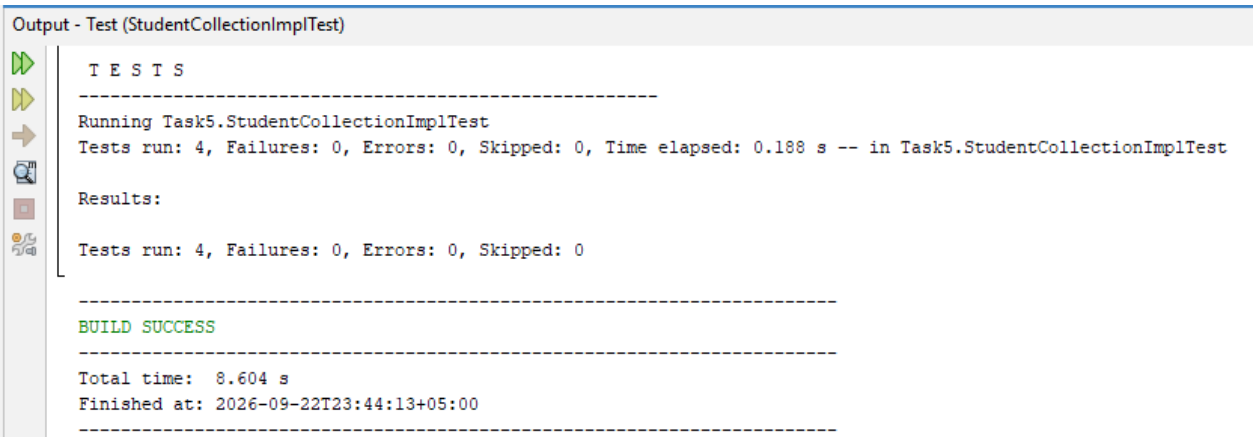
● StudentCollectionImpl.java, (Java Class):

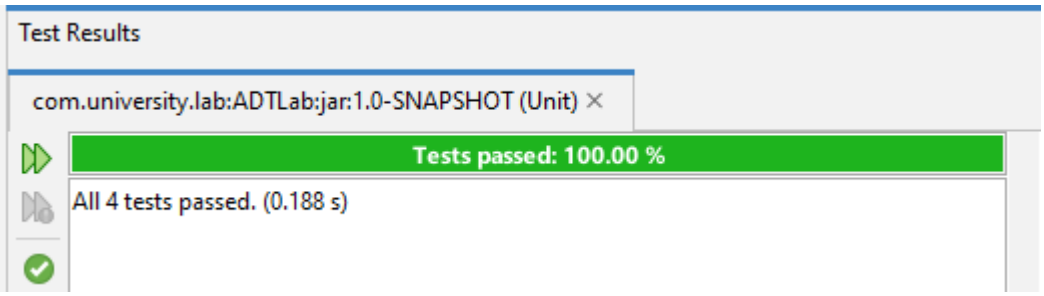
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/main/java/Task5/StudentCollectionImpl.java

● StudentCollectionImplTest.java, (Junit test class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab6/src/test/java/Task5/StudentCollectionImplTest.java

OUTPUT:





CONCLUSION:

This lab provided hands-on experience in core software construction principles, including abstraction, encapsulation, and interface-driven design. By separating specifications from implementations across custom ADTs and proving system behavior through JUnit testing, I reinforced best practices for building modular, maintainable, and robust Java applications.